



STUDENT

0017-RRW

TENTAMEN

250118 DV2606 2110

Salstentamen (DI)

Kurskod	--
Bedömningsform	--
Starttid	18.01.2025 11:00
Sluttid	18.01.2025 16:00
Bedömningsfrist	--
PDF skapad	05.11.2025 17:12
Skapad av	Anders Göransson

- 1.1** You are parallelizing a sequential program. Assume that you manage to parallelize 95% of the sequential program, i.e., 5% is still executed sequentially. What is the highest speedup that you can archive (given an infinite number of processors)?

Select one alternative:

☐ None of the other alternatives is correct

☐ 5

☐ Infinite

☐ 95

☒ 20



Rätt. 1 av 1 poäng.

- 1.2** Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *decrease* the efficiency?

☐ Increase the clock frequency of the processors.

☐ Increase the number of processors.

☐ Put more barrier synchronizations in the code.

☐ Decrease the input data set size.

☒ All alternatives decrease the efficiency.



Rätt. 1 av 1 poäng.

1.3 *SPMD* is a common programming model for writing parallel programs. *SPMD* stands for:

- ☐ Single Processor Multiple threads
- ☐ Shared-memory Programming for Multiple Data
- ☐ None of the alternatives is correct.
- ☐ Shared-memory Program Many Distributions
- ☒ Single Program Multiple Data



Rätt. 1 av 1 poäng.

1.4 Simultaneous multithreading is ...

- ☒ a technique to exploit parallelism in the instruction execution pipeline
- ☐ a technique to improve branch prediction
- ☐ a software approach to parallelize an application
- ☐ none of the alternatives is correct
- ☐ a technique to implement multi-level caches



Rätt. 1 av 1 poäng.

1.5 Why are snoopy-based cache coherence protocols not used in large-scale shared-memory multiprocessors?

- ☐ They can not guarantee coherence for more than ten caches.
- ☐ They can only be implemented using a write-update policy.
- ☐ None of the alternatives is correct.
- ☒ They are not scalable to a large number of nodes. 
- ☐ Modern processors are too fast for snoopy-based protocols.

Rätt. 1 av 1 poäng.

1.6 MSI (Modified-Shared-Invalid) is a basic write-invalidate protocol. Assuming that a particular cache block is in state Modified, which action is performed when the cache sees a read request on the shared bus?

- ☐ The request is denied since the block is in state Modified.
- ☒ The cache replies with the requested block, and the block stays in state Modified. 
- ☐ None of the alternatives is correct. 
- ☐ The processor is always stalled until the bus request is handled.
- ☐ The cache replies with the requested block, and the block is marked as Invalid.

Fel. 0 av 1 poäng.

1.7 A fundamental difference between a directory-based cache coherence protocol and a snoopy-based cache coherence protocol is ...

- ☒ A directory-based protocol keeps exact information in which caches copies of each memory block reside. 
- ☐ A directory-based protocol relies on a shared broadcast medium.
- ☐ A directory-based protocol can only be implemented using the write-update policy.
- ☐ A directory-based protocol can only be implemented using the write-invalidate policy.
- ☐ All of the alternatives are correct.

Rätt. 1 av 1 poäng.

1.8 What is the (asymptotic) parallel time complexity of *matrix-matrix multiplication* using n processors, assuming $n \times n$ matrices?

- ☐ $O(n)$ 
- ☐ None of the alternatives is correct.
- ☐ $O(n^2)$ 
- ☐ $O(n \log n)$
- ☐ $O(\log n)$

Fel. 0 av 1 poäng.

1.9 A PRAM (Parallel Random Access Machine) can be useful for ...

- ☐ Running applications with very large input data sets.
- ☐ Running message-passing applications with GPU support.
- ☐ Very easy implementation of parallel hardware.
- ☒ Designing and evaluating parallel algorithms.
- ☐ All of the alternatives are correct.



Rätt. 1 av 1 poäng.

2.1 Describe *Flynn's taxonomy* of different computer architectures.

Flynn's taxonomy is a way to classify different computer architectures. The classes are:

SISD (Single Instruction Single Data): This is the architecture of the first computers.

SIMD (Single Instruction Multiple Data): This architecture can be found in the threads of GPUs where each thread can only do one instruction at a time.

MISD (Multiple Instruction Single Data): This is more a theoretical architecture that could be used in pipelines

MIMD (Multiple Instruction Multiple Data): This is the architecture that is implemented in CPUs.

Ord: 83

Besvarad.

- 2.2** Shared-memory multiprocessors exist in two different variants: UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) multiprocessors. Describe the primary difference between them, and also give an example when one is preferable over the other.

UMA means that each possessing core reaches all data in a uniform time span. This is preferable on small systems where this uniform time can be kept under a certain threshold.

NUMA means that the time it takes for two different processors to reach the same data can differ. This is because the processors are located at different lengths from the data. Normally there is some memory kept close to each processing core. This means that some data is quick to access and some takes longer. This is preferable when we need to have a scalable system since new cores and memory can be easily attached.

Ord: 106

Besvarad.

- 2.3** When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.

- Describe the main characteristics of each of the two approaches.
- Further, describe some advantage and disadvantage with each of them.
- Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

Task parallelism is when we parallelize for example one task when we need to do one certain calculation and another task when we need to do a different calculation. This sort of parallelism needs synchronization to work well which is a disadvantage. One application that is suitable for task parallelism is for example a video editor where I might want to play the video and add a filter at the same time.

Data parallelism is when we have one task that is a large calculation. Then we can parallelize the calculations over different parts of the data. Disadvantages with this is that we need to control what thread reads and writes from what data and that there are no race conditions. Applications for this could be large matrix calculations where a sequential program would do many iterative tasks and a parallel program would split these tasks and take advantage of all resources in the processor.

Ord: 155

Besvarad.

- 2.4** When we parallelize a sequential application, e.g., by using threads or message-passing, so it can execute in parallel on N processors, we often find that the parallel program is not N times faster. Describe two important reasons for that.

Most of the time there are parts of the program that we are not able to parallelize. These sequential parts take time and then the total time of the program is the sequential time plus the parallel time which means that the program will not be N times faster.

One other reason is that we may need to use some parallel synchronization like barriers. This will also add time to the overall program and it will not be N times faster.

There is also overhead when we create the threads.

Ord: 90

Besvarad.

- 2.5** Sometimes we find that a system or application has a *superlinear speedup*. Clearly describe two situations when and why it may occur.

Sometimes when we parallelize a program the caches are more efficient for the threads than when we had a sequential program which can make the parallel program more than N times faster and we then have a superlinear speedup.

This is also possible when the task becomes smaller when we parallelize it. One example of this is binary search when the sequential program needs to search through each branch which takes time. The parallel version then lets each thread search one branch giving each thread less to search resulting in a superlinear speedup.

Ord: 92

Besvarad.

- 2.6** *Cache coherence* is an important issue to handle in a shared-memory multiprocessor. Describe, by giving an example, why ensuring cache coherence is important. (1p)
Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)
Describe some advantage and disadvantage with each of the approaches. (1p)

When different threads work on the same data they can sometimes write over data that another thread has cached. The other threads cache is then wrong which can cause serious problems in further calculations.

Write-invalidate is when a thread writes to the memory then invalidates all caches that points to that memory.

With write-update instead of invalidating the caches the data is instead updated in each cache the points to that memory.

Advantages of write-invalidate is that it is faster and of the data that is invalidated is not used very often the time it takes to cache the data again after it has been invalidated is less than what it would cost to update each cach. However if the data is used often this will cause many cache faults.

Advantages of write-update is that if the data is used often this method will cause the caches to work efficiently. If the data is not used often however, the time that it takes to update all caches will be unnecessary and make the program run slower.

Ord: 176

Besvarad.

3.1 Thread coarsening

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ Thread coarsening decreases the completion time of a thread
- ☒ Thread coarsening can make it possible to use the shared memory more efficiently in a CUDA program 
- ☐ Thread coarsening increases the parallelism in a CUDA program
- ☐ Thread coarsening makes it possible to perform data transfer between the CPU and GPU in parallel with kernel execution on the GPU
- ☒ Thread coarsening can reduce the number redundant calculations in a CUDA program 

Rätt. 2 av 2 poäng.

3.2 Reduction sum

Consider the two functions SumReduction1 and SumReduction2 below:

```
__global__ void SumReduction1(float* input) {
    int i = 2*threadIdx.x
    for (int stride = 1; stride <= blockDim.x; stride *= 2) {
        if (threadIdx.x % stride == 0) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}

__global__ void SumReduction2(float* input) {
    int i = threadIdx.x
    for (int stride = blockDim.x; stride >= 1; stride /= 2) {
        if (threadIdx.x < stride) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}
```

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ SumReduction1 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction2.
- ☒ SumReduction2 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction1. 
- ☐ SumReduction1 uses a gather approach and SumReduction2 uses a scatter approach.
- ☐ SumReduction1 has less control divergence than SumReduction2.
- ☒ SumReduction2 has less control divergence than SumReduction1 

Rätt. 2 av 2 poäng.

3.3

Consider the following small program.

```
for (int m = 0; m < M; m++) {  
    input[m].value += 5;  
    for (int n = 0; n < N; n++) {  
        output[n].value += input[m].value / distance(input[m].position, output[n].position);  
    }  
}
```

If we transform this program into a parallel CUDA program, we may run into some serious performance problems in the CUDA program.

What kind of performance problems may the CUDA program suffer from? How could we modify the small program above so that it is still functionally equivalent and so that the corresponding CUDA program does not suffer from any serious performance problems?

This program is gather to scatter which requires us to use atomic operations that will tank our performance. We want to have the for-loop that iterates over N first and then iterate over M inside if it. However, since we have computations in the M loop, before N, we will need to run this before our other loop. The program would look something like this:

```
for (int m = 0; m < M; m++) {  
    input[m].value += 5;  
}  
for (int n = 0; n < N; n++) {  
    for (int m = 0; m < M; m++) {  
        output[n].value += input[m].value / distance(input[m].position, output[n].position);  
    }  
}
```

Ord: 106

Besvarad.

3.4

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☒ A CPU normally has larger caches than a GPU. 
- ☐ The processor cores on a CPU are normally organized in streaming multiprocessors.
- ☐ The processor cores in a CPU normally execute in SIMD mode.
- ☒ Context switching on a GPU is normally faster than context switching on a CPU. 
- ☐ A CPU normally has more processor cores than a GPU.

Rätt. 2 av 2 poäng.

3.5

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ The CPU can access variables in GPU shared memory
- ☐ All the threads in a thread block see the same registers
- ☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 
- ☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.
- ☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 

Rätt. 2 av 2 poäng.

3.6 Cuda Streams

What are Cuda Streams? How can they be used to improve performance? Give a short example.

Cuda streams is a way to take advantage of the buss in GPUs we can use Cuda streams to send data to the CPU at the same time that we compute data on the GPU. This can be used to improve performance when we work on large data and one part of the data is done then we can sent it while we work in the next part instead of waiting until all parts are done and then sending everything which takes longer.

Ord: 83

Besvarad.

3.7 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☒ A Thread Block can contain many Warps.



☐ Threads in the same Warp can execute on different Streaming Multiprocessors.

☒ Threads in the same Warp cannot execute different instructions at the same time.



☐ Threads from different Thread Blocks can execute in the same Warp.

☐ A Thread Block cannot be larger than a Warp.

Rätt. 2 av 2 poäng.

3.8 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Privatization is when a thread does calculations on variables in its own registries and send writes to global data when it is done. This is faster than to write directly to global data since then we would have to use atomic operations in order to not have race conditions when two threads tries to write to the same data. Privatization strongly decreases the amount of atomic operations needed.

Ord: 68

Besvarad.

3.9 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use `__syncthreads()` when using tiling.

Tiling is when we split the data into smaller tiles that can be worked on separately. This is good when we have the same calculations that needs to be done for each thread. Then we can tile it, load the data to shared memory, and do the calculations. The calculations can then be accessed by all threads in the same block and will only have to be calculated ones. At the same time it is much faster for threads to access data from shared memory over global memory which will improve performance. We need to use `syncthreads()` in order to make threads wait untill all the treads in the block is done before moving on to the next tile.

Ord: 119

Besvarad.

3.10

Occupancy is defined as the ratio of active warps on an SM to the maximum number of active warps supported by the SM. Why do we normally want to have high occupancy? Which hardware related parameters affect occupancy? Discuss and give examples of how we as programmers, by taking smart design decisions, can provide good trade-offs between the parameters that affect occupancy.

High occupancy means that we are using all available resources to do our task. Hardware related parameters like "max threads per block" and "max blocks" affect occupancy. Occupancy can be affected by choosing an appropriate amount of blocks and threads per block to be used by the kernel. By having more threads we give ourselves more threads that can access shared memory which is faster than global. But if we have too many threads there will be less room for shared memory. It is a balance that is task specific that has to be made.

Ord: 95

Besvarad.